

Chapter 12

Kubernetes in Action 1st Edition

Securing the Kubernetes API Server

인증과 RBAC 인가 — 누가, 무엇을 할 수 있는가



신재근 (Jagger)

✓ 실측 검증됨 · minikube v1.38.1 · K8s v1.35.1 · 2026-05-28

왜 12장인가

11장에서 본 API Server 파이프라인

Authentication → Authorization → Admission → Validation → etcd

12장이 답하는 두 가지 질문

- 누가 호출했는가? — 인증 (Authentication)
- 무엇을 할 수 있는가? — 인가 (Authorization / RBAC)

가장 자주 발생하는 클러스터 사고의 80% 이상이 이 두 단계의 설정 미숙에서 시작됩니다.

12장의 두 축

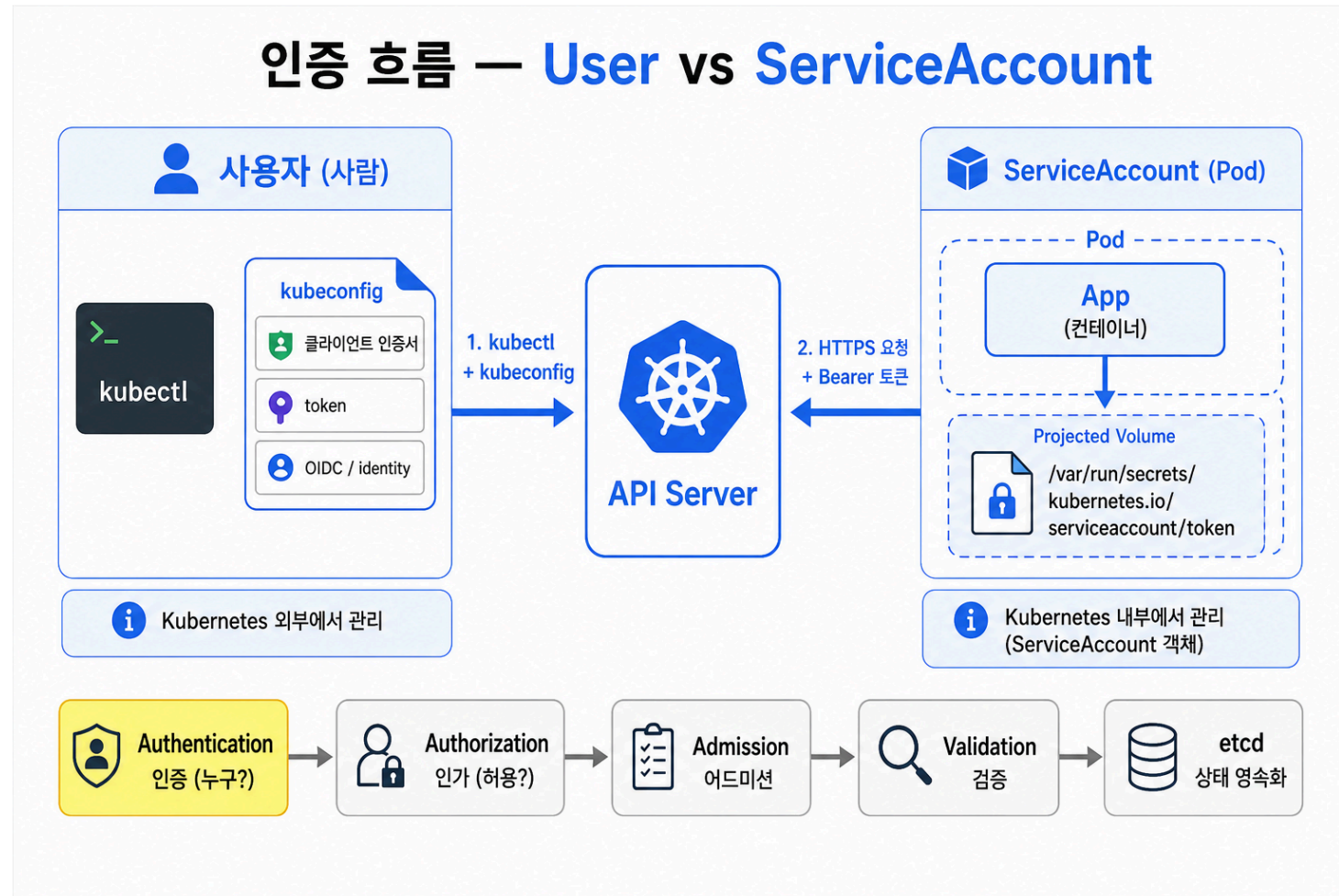
12.1 Authentication

- User vs ServiceAccount의 차이
- Pod이 API Server를 호출할 때 토큰이 어디서 오는가
- 책 1판 vs 현행: Projected Token으로 진화

12.2 RBAC (Role-Based Access Control)

- Role / RoleBinding / ClusterRole / ClusterRoleBinding 4종 매트릭스
- 최소 권한 원칙과 실패 사례

12.1 Authentication 개요



User vs ServiceAccount — 핵심 차이

항목	User	ServiceAccount
용도	사람/외부 시스템	Pod 내부 프로세스
관리 주체	K8s 밖 (kubeconfig, OIDC, IAM)	K8s 안 (ServiceAccount 리소스)
<code>kubectl get</code> 가능?	불가	가능
토큰 출처	외부 발급	API Server가 발급
네임스페이스	없음	있음

핵심 한 줄

User는 K8s가 모르는 신원, SA는 K8s가 만든 신원

default ServiceAccount의 함정

모든 네임스페이스에 자동 생성

- 새 네임스페이스 만들면 `default` 라는 SA가 자동 생성됨
- Pod에 `serviceAccountName` 을 지정하지 않으면 → `default` SA가 자동 부착
- 즉, 모든 Pod은 항상 어떤 SA를 가진다 (없는 Pod은 존재하지 않음)

운영에서 자주 보이는 사고

누군가 `default` SA에 `cluster-admin` ClusterRole을 바인딩 → 모든 Pod이 클러스터 전체 권한 보유

권장 패턴

- 워크로드마다 전용 SA를 만들어 부착
- 토큰이 불필요한 Pod엔 `automountServiceAccountToken: false`

Pod에 토큰이 들어오는 경로 (책 1판)

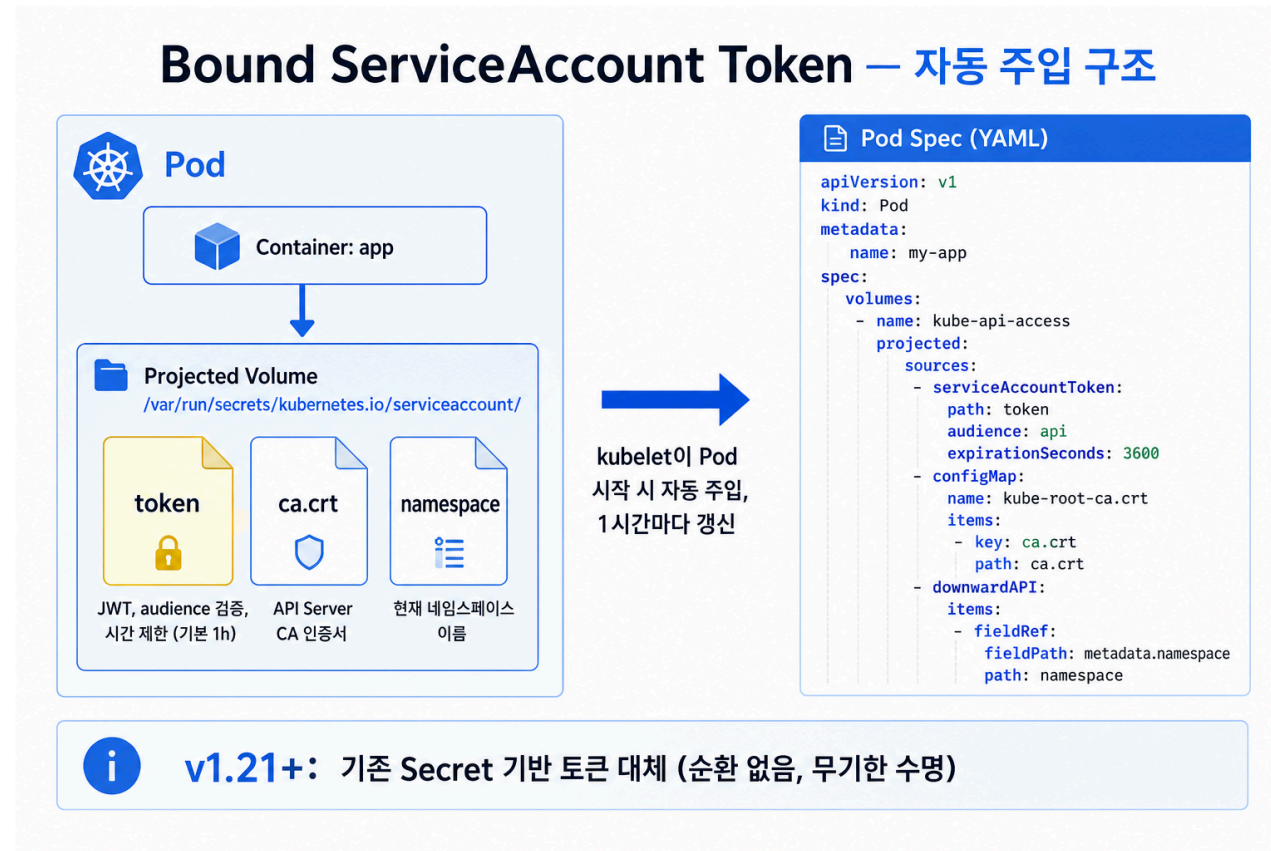
책의 설명

- ServiceAccount 생성 시 자동으로 `Secret` 이 생성됨
- Secret 안에 무제한 수명의 토큰(JWT)이 들어있음
- Pod에 SA가 부착되면 kubelet이 Secret을 마운트
- `/var/run/secrets/kubernetes.io/serviceaccount/token` 파일로 접근

1판 시점의 특징

- 토큰 수명: 무제한 (revoke 안 하면 평생 유효)
- 토큰 회전: 없음
- audience: 없음 (모든 청중에게 유효)

Pod에 토큰이 들어오는 경로 (현행 1.21+)



책 1판 vs 현행 — 토큰 진화

항목	책 1판 (2018)	현행 (1.24+)
토큰 저장	Secret 객체	Pod에 직접 projected 마운트
토큰 수명	무제한	기본 1시간, 자동 회전
audience	없음	지정 가능
SA 생성 시 Secret	자동 생성	자동 생성 안 함 (수동 요청 시만)
명령	<code>kubectl get secret <sa>-token-xxx</code>	<code>kubectl create token <sa></code>

시연 명령 (현행)

```
kubectl create token foo --duration=3600s
kubectl create token foo --audience=api
```

Demo 1 (1/4) — ServiceAccount 생성

명령 + 실측 출력

```
$ kubectl apply -f serviceaccount.yaml
serviceaccount/foo created

$ kubectl describe sa foo
Name:                foo
Namespace:           default
Labels:              <none>
Annotations:         <none>
Image pull secrets:  <none>
Events:              <none>
```

책과 다른 점 (실측)

- 책의 `Mountable secrets:` / `Tokens:` 필드가 출력 자체에 없음 → 1.24+ 정상
- `foo-token-xxxxx` Secret도 생성되지 않음

Demo 1 (2/4) — Pod에 SA 부착

curl-pod-sa.yaml 핵심

```
spec:
  serviceAccountName: foo
  volumes:
  - name: kube-api-access
    projected:
      sources:
      - serviceAccountToken:
          expirationSeconds: 3600
          path: token          # audience 미명시 → 기본(API Server)
```

```
$ kubectl apply -f curl-pod-sa.yaml && kubectl wait --for=condition=ready pod/curl-custom-sa
pod/curl-custom-sa created
pod/curl-custom-sa condition met
```

```
$ kubectl get pod curl-custom-sa
```

NAME	READY	STATUS	RESTARTS	AGE
curl-custom-sa	1/1	Running	0	6s

Demo 1 (3/4) — Pod 안에서 토큰 확인

명령 + 실행 출력

```
$ kubectl exec curl-custom-sa -- ls /var/run/secrets/kubernetes.io/serviceaccount/  
ca.crt  
namespace  
token  
  
$ kubectl exec curl-custom-sa -- sh -c 'cat /var/run/secrets/kubernetes.io/serviceaccount/token | head -c 60'  
eyJhbGciOiJSUzI1NiIsImtpZCI6InA4UEYxOFJyb185Ry1YQW9QZUhUQmZE  
  
$ kubectl exec curl-custom-sa -- sh -c 'wc -c /var/run/.../token'  
1069 /var/run/secrets/kubernetes.io/serviceaccount/token    # JWT 길이
```

관찰

- 3개 파일(token/ca.crt/namespace) 모두 projected volume으로 마운트
- token은 1069바이트 JWT, 기본 audience(API Server) — 1시간 후 자동 회전

Demo 1 (4/4) — API Server 직접 호출

명령 (Pod 안에서)

```
kubectl exec curl-custom-sa -- sh -c '
TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
CACERT=/var/run/secrets/kubernetes.io/serviceaccount/ca.crt
curl -s -w "HTTP=%{http_code}\n" --cacert $CACERT \
  -H "Authorization: Bearer $TOKEN" \
  https://kubernetes.default.svc/api/v1/namespaces/default/pods'
```

실패 결과 (RBAC 부여 전)

```
{
  "kind": "Status", "status": "Failure", "reason": "Forbidden", "code": 403,
  "message": "pods is forbidden: User \"system:serviceaccount:default:foo\"
              cannot list resource \"pods\" in API group \"\" in the namespace \"default\""
}
```

HTTP=403

인증은 통과(401 아님). 인가에서 차단(403). ← RBAC가 필요한 이유

12.2 RBAC — 왜 필요한가

401 vs 403 — 헛갈리지 말 것

코드	의미	해결 책임
401 Unauthorized	신원 확인 실패	인증 설정 (토큰/인증서)
403 Forbidden	신원은 OK, 권한 부족	RBAC 설정

12.2의 목표

"system:serviceaccount:default:foo가 pods를 list할 수 있게 해줘"

이 한 줄을 어떤 K8s 객체로 표현하는가가 12.2의 전부입니다.

RBAC 4종 리소스 매트릭스

RBAC 4가지 리소스 — Matrix		
	무엇 (권한 정의)	누가 → 무엇 (바인딩)
네임스페이스 단위	Role ONE 네임스페이스에서 권한(verbs + resources) 정의 get/list/watch pods in 'default' ns	RoleBinding Subject(User/Group/SA)를 Role에 바인딩 (네임스페이스 한정) ServiceAccount 'foo' → Role 'pod-reader'
클러스터 단위	ClusterRole 전체 네임스페이스 또는 클러스터 범위 리소스에 권한 정의 list nodes, list PVs, list ClusterRoles	ClusterRoleBinding Subject를 ClusterRole에 바인딩 (전체 네임스페이스) Group 'admins' → ClusterRole 'cluster-admin'
 Trick: ClusterRole + RoleBinding = 여러 네임스페이스에서 같은 역할 재사용		

Role의 구성 요소

YAML 한 줄씩 해석

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default      # 이 Role이 적용되는 네임스페이스
  name: pod-reader
rules:
- apiGroups: ["" ]        # core API group ("" = pods/services 등)
  resources: ["pods"]      # 어떤 리소스 종류
  verbs: ["get","list","watch"] # 어떤 동작
```

핵심: 3차원 권한 (verb × resource × apiGroup)

- Verb: `get` / `list` / `watch` / `create` / `update` / `patch` / `delete` (실무 7~8개)
- Resource: `pods`, `services`, `deployments`, `secrets` 등
- apiGroup: core는 `""`, Deployment는 `apps`, Job은 `batch`

RoleBinding — Subject를 Role에 묶기

YAML 구조

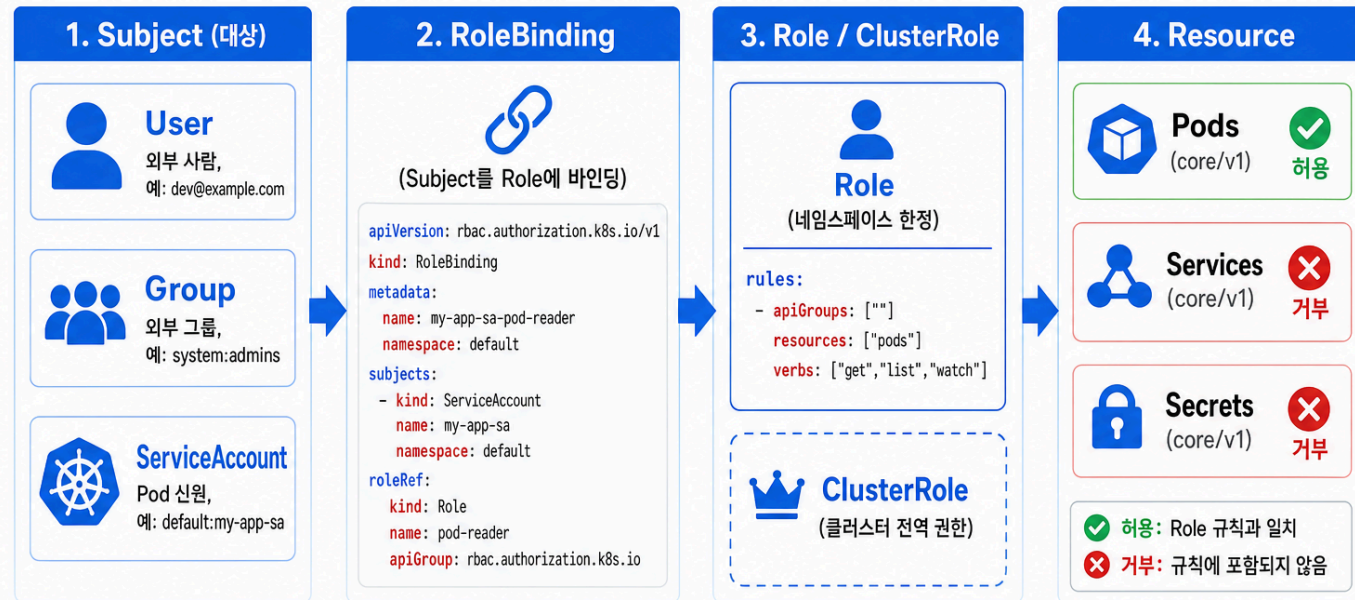
```
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
- kind: ServiceAccount      # 또는 User, Group
  name: foo
  namespace: default
roleRef:
  kind: Role                # 또는 ClusterRole
  name: pod-reader
  apiGroup: rbac.authorization.k8s.io
```

핵심 규칙

- `subjects` 는 여러 명 지정 가능 (배열), `roleRef` 는 한 개 (단일 객체)
- `roleRef` 는 immutable — 역할 바꾸려면 RoleBinding 자체를 새로 생성

Subjects 3종

RBAC: Subject → Binding → Role → Resource (4단계 흐름)



Authorization은 Admission보다 먼저 실행.
일치하는 binding 없음 = 403 Forbidden.

Default Deny — RBAC의 핵심 철학

명시되지 않은 모든 권한은 거부

- 권한 부여는 반드시 명시적으로 (RoleBinding/ClusterRoleBinding)
- 부정형 권한 부여 (deny 규칙) 없음 — 화이트리스트만 존재
- 권한 합집합: 여러 RoleBinding에 매칭되면 권한이 합쳐짐

실용 의미

"방금 SA에 권한을 추가했는데도 안 돼요" → 매칭되는 binding이 정말 있는지 다시 확인

```
kubectl auth can-i list pods --as=system:serviceaccount:default:foo
```

Demo 2 (1/5) — Role 생성

pod-reader-role.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: default
  name: pod-reader
rules:
- apiGroups: [""]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

```
kubectl apply -f pod-reader-role.yaml
kubectl describe role pod-reader -n default
```


Demo 2 (2/5) — RoleBinding 생성

pod-reader-rolebinding.yaml

```
kind: RoleBinding
metadata:
  name: pod-reader-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: foo
  namespace: default
roleRef:
  kind: Role
  name: pod-reader
```

```
kubectl apply -f pod-reader-rolebinding.yaml
kubectl describe rolebinding pod-reader-binding
```

Demo 2 (3/5) — 권한 사전 검증

적용 전후 실측

```
SA=system:serviceaccount:default:foo

$ kubectl auth can-i list pods --as=$SA          # 적용 전
no
$ kubectl apply -f pod-reader-rolebinding.yaml
$ kubectl auth can-i list pods --as=$SA          # 적용 후
yes
$ kubectl auth can-i list pods --as=$SA -n kube-system # 다른 ns
no                                                    # Role의 ns 한정
```

권한 부여 전에 `--as` 로 시뮬레이션 — 사고 예방 1순위

Demo 2 (4/5) — Pod 안에서 실제 호출

```
kubectl exec curl-custom-sa -- sh -c '
  TOKEN=$(cat /var/run/secrets/kubernetes.io/serviceaccount/token)
  curl -s -w "HTTP=%{http_code}\n" \
    --cacert /var/run/secrets/kubernetes.io/serviceaccount/ca.crt \
    -H "Authorization: Bearer $TOKEN" \
    https://kubernetes.default.svc/api/v1/namespaces/default/pods | jq "{kind,items_count:(.items|length)}"
'
```

실행 결과

```
{ "kind": "PodList", "items_count": 4 }
HTTP=200
```

같은 Pod의 같은 curl이 Demo 1에선 403 → 지금 200. RoleBinding 하나 차이.

Demo 2 (5/5) — 네임스페이스 경계 확인

kube-system의 pods는 못 본다

```
kubectl exec curl-custom-sa -- sh -c '
  TOKEN=$(cat /var/run/.../token)
  curl --cacert .../ca.crt \
    -H "Authorization: Bearer $TOKEN" \
    https://kubernetes.default.svc/api/v1/namespaces/kube-system/pods
'
# → 403 Forbidden
```

왜?

- Role과 RoleBinding 모두 namespace: default
- 다른 네임스페이스에는 효력 없음 → 네임스페이스 격리

Demo 3 (1/5) — ClusterRole 생성

service-lister-clusterrole.yaml

```
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRole
metadata:
  name: service-lister
rules:
- apiGroups: [""]
  resources: ["services"]
  verbs: ["get", "list", "watch"]
```

```
kubectl apply -f service-lister-clusterrole.yaml
kubectl get clusterrole service-lister
```

Demo 3 (2/5) — ClusterRoleBinding 생성

service-lister-clusterrolebinding.yaml

```
kind: ClusterRoleBinding
metadata:
  name: service-lister-binding
subjects:
- kind: ServiceAccount
  name: foo
  namespace: default
roleRef:
  kind: ClusterRole
  name: service-lister
```

```
kubectl apply -f service-lister-clusterrolebinding.yaml
kubectl describe clusterrolebinding service-lister-binding
```

Demo 3 (3/5) — 모든 네임스페이스에 효력

모든 네임스페이스의 services를 list

```
SA=system:serviceaccount:default:foo
for ns in default kube-system kube-public; do
  echo -n "$ns" services: "
  kubectl auth can-i list services --as=$SA -n $ns
done
```

실행 결과

```
[default] services: yes
[kube-system] services: yes
[kube-public] services: yes
```

```
# 실제 API 호출 (외부 토큰)
[default] HTTP=200 count=3
[kube-system] HTTP=200 count=1
[kube-public] HTTP=200 count=0
```

Demo 3 (4/5) — 권한은 합쳐진다

현재 SA `foo`의 권한 상태

- RoleBinding `pod-reader-binding` → default 네임스페이스 pods 읽기
- ClusterRoleBinding `service-lister-binding` → 전 클러스터 services 읽기

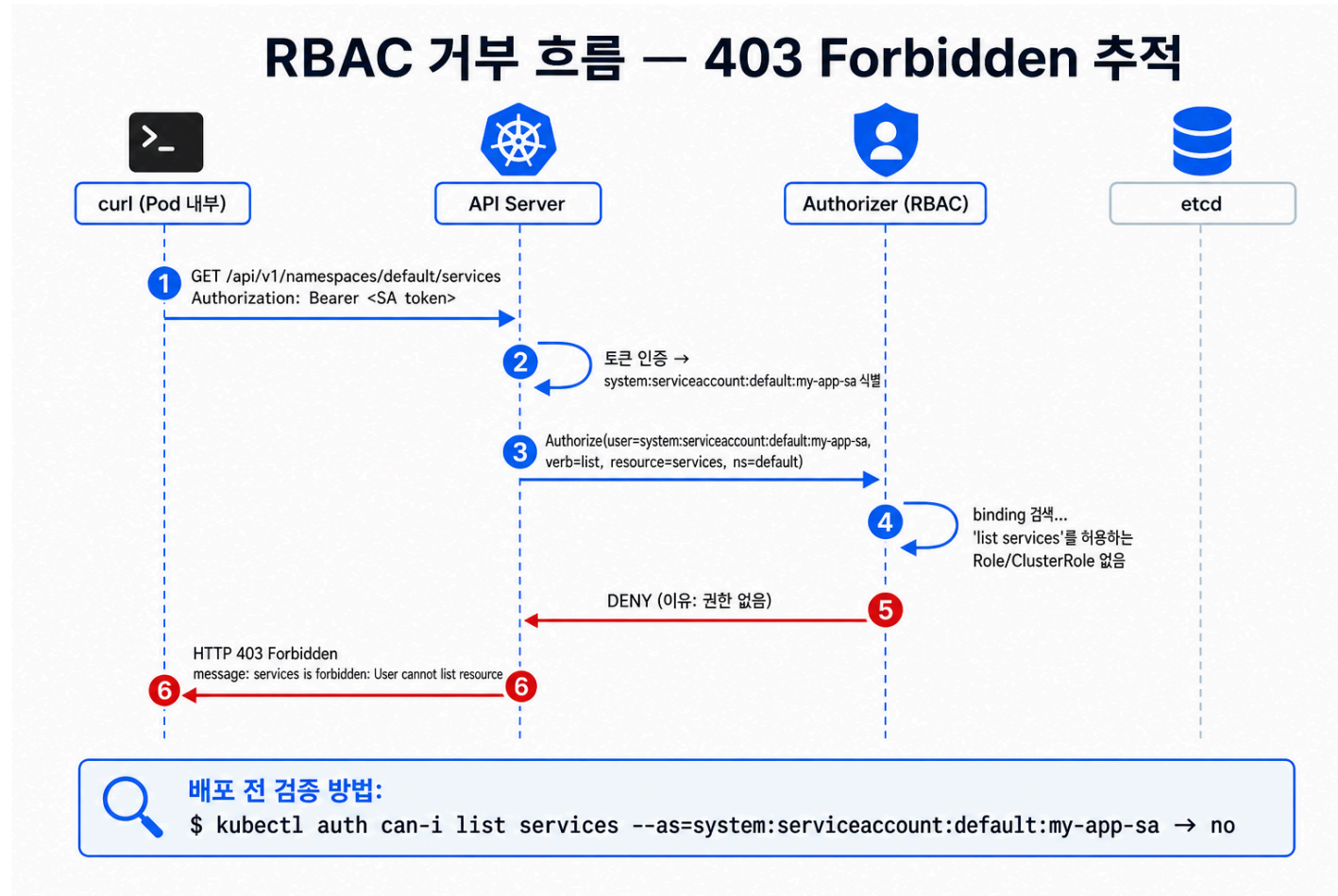
확인

```
kubectl auth can-i list pods -n default \
  --as=system:serviceaccount:default:foo
# → yes (RoleBinding으로 인해)
```

```
kubectl auth can-i list services -n kube-system \
  --as=system:serviceaccount:default:foo
# → yes (ClusterRoleBinding으로 인해)
```

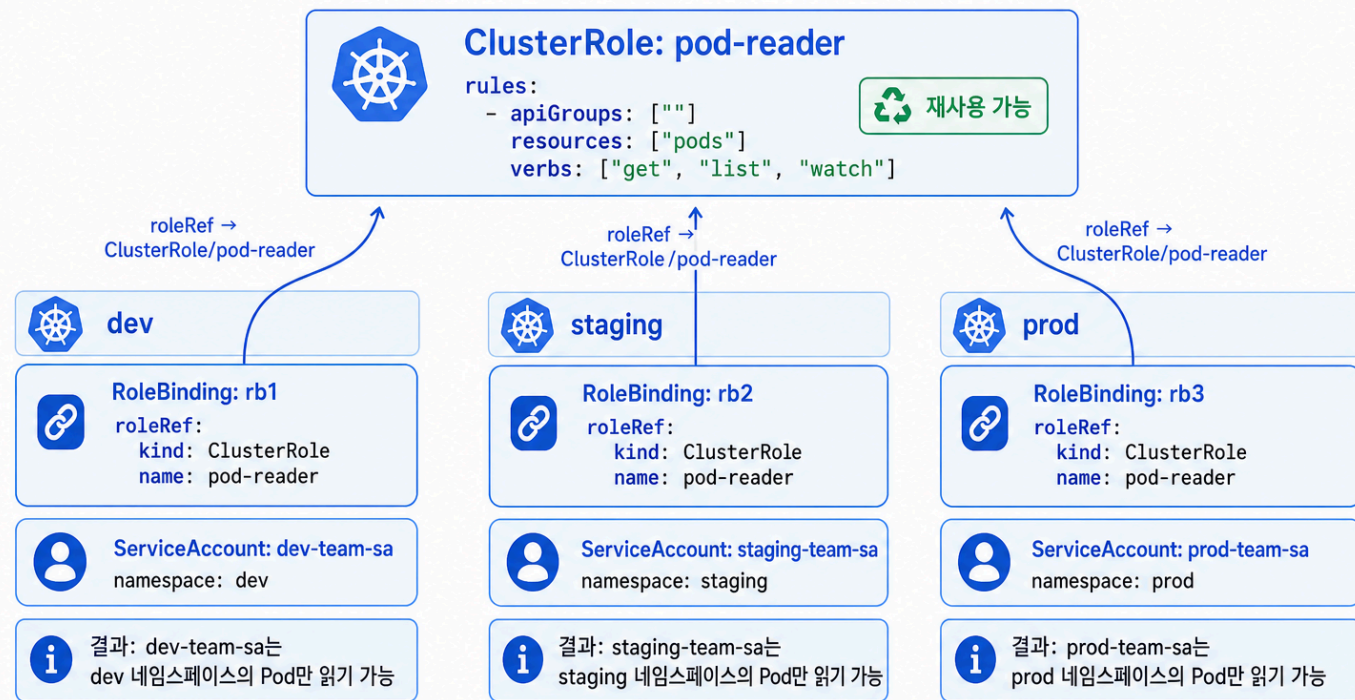
```
kubectl auth can-i delete pods -n default \
  --as=system:serviceaccount:default:foo
# → no (그 권한은 어디에도 없음)
```


Demo 3 (5/5) — 권한 거부 트레이스



트릭 — ClusterRole + RoleBinding

ClusterRole + RoleBinding = 여러 네임스페이스에서 권한 재사용



원리: ClusterRole이 권한을 전역 정의, RoleBinding이 자기 네임스페이스로 범위 제한.
중복 Role 객체 제거.

ClusterRole + RoleBinding — 동작 규칙

핵심 규칙

조합	권한 정의 범위	권한 효과 범위
Role + RoleBinding	한 네임스페이스	RoleBinding의 네임스페이스
ClusterRole + RoleBinding	클러스터 전체 정의	RoleBinding의 네임스페이스만
ClusterRole + ClusterRoleBinding	클러스터 전체 정의	클러스터 전체

실용 시나리오

built-in `view`, `edit`, `admin` ClusterRole을 각 팀 네임스페이스에 RoleBinding으로 묶기

```
roleRef:
  kind: ClusterRole
  name: view      # built-in
  apiGroup: rbac.authorization.k8s.io
```

Demo 4 (1/3) — built-in view 재사용

view-namespace-rolebinding.yaml

```
kind: RoleBinding
metadata:
  name: view-default-binding
  namespace: default
subjects:
- kind: ServiceAccount
  name: foo
  namespace: default
roleRef:
  kind: ClusterRole
  name: view          # K8s 기본 제공
  apiGroup: rbac.authorization.k8s.io
```

```
kubectl apply -f view-namespace-rolebinding.yaml
```

Demo 4 (2/3) — view 권한 확인

```
SA=system:serviceaccount:default:foo

$ kubectl auth can-i get pods -n default --as=$SA
yes

$ kubectl auth can-i get secrets -n default --as=$SA
no                                     # ← view는 Secret 못 봄!

$ kubectl auth can-i create pods -n default --as=$SA
no                                     # ← view는 읽기만

$ kubectl auth can-i get pods -n kube-system --as=$SA
no                                     # ← RoleBinding이라 default만
```

view의 4가지 특성: 읽기 가능 / Secret 불가 / 변경 불가 / ns 격리

Demo 4 (3/3) — 권한 정리

데모용 리소스 삭제

```
kubectl delete rolebinding pod-reader-binding
kubectl delete rolebinding view-default-binding
kubectl delete clusterrolebinding service-lister-binding
kubectl delete role pod-reader
kubectl delete clusterrole service-lister
kubectl delete pod curl-custom-sa
kubectl delete sa foo
```

최소 권한 원칙 (Principle of Least Privilege)

운영 환경의 SA는 모든 데모 끝나면 권한도 정리하는 습관

Default ClusterRoles



기본 ClusterRole — 내장 RBAC 역할

기능	view	edit	admin	cluster-admin
 리소스 읽기	✓	✓	✓	✓
 리소스 수정	✗	✓	✓	✓
 Secret 읽기	✗	✗	✓	✓
 RBAC 관리 (Role/Binding)	✗	✗	✓ (네임스페이스 한정)	✓
 클러스터 범위 리소스 (Node, PV)	✗	✗	✗	✓
 적용 범위	네임스페이스	네임스페이스	네임스페이스	클러스터 전체

**view**
사용 사례:
읽기 전용 대시보드,
모니터링, 감사

**edit**
사용 사례:
개발자가 직접
워크로드 배포

**admin**
사용 사례:
네임스페이스 소유자가
본인 네임스페이스 전체 관리

**cluster-admin**
사용 사례:
플랫폼 팀 전용 —
신중히 사용!

 **cluster-admin을 ServiceAccount에 바인딩하지 말 것. 보안 사고의 대부분이 여기서 시작됩니다.**

system: ClusterRoles — 보지 말 것

kube-system이 쓰는 내부 ClusterRole

```
kubectl get clusterrole | grep ^system:  
# system:auth-delegator  
# system:controller:deployment-controller  
# system:kube-controller-manager  
# system:kube-scheduler  
# system:node  
# ... 수십 개
```

규칙

- **system:** 접두어 ClusterRole은 K8s 내부 컴포넌트용
- 직접 RoleBinding으로 사용자에게 부여하지 말 것
- 자동으로 시스템 SA에 바인딩되어 있음 → 변경 금지

Demo 5 (1/3) — 권한 시뮬레이션 도구

kubectl auth can-i

```
# 내가 할 수 있는 것 확인
kubectl auth can-i create deployments
kubectl auth can-i delete nodes
kubectl auth can-i '*' '*' --all-namespaces

# 다른 사용자/SA로 시뮬레이션
kubectl auth can-i list pods \
  --as=system:serviceaccount:default:foo

# 그룹으로 시뮬레이션
kubectl auth can-i create services \
  --as=alice --as-group=system:authenticated
```

Demo 5 (2/3) — 어디서 권한이 오는가

실측 — SA가 가진 모든 권한 (default ns)

```
$ kubectl auth can-i --list --as=system:serviceaccount:default:foo
Resources          Verbs
selfsubjectaccessreviews.auth... [create]
configmaps          [get list watch]
pods                [get list watch] ← Role+RoleBinding
pods/log            [get list watch]
services            [get list watch]
deployments.apps    [get list watch]
... (30+ resources)
```

view ClusterRole이 30+ 리소스에 read 권한을 한 번에 부여 — 합집합 모델

Demo 5 (3/3) — 위험한 binding 찾아내기

운영 점검 스크립트

```
# cluster-admin을 SA에 바인딩한 것 찾기
kubectl get clusterrolebindings -o json | jq '
  .items[] | select(.roleRef.name == "cluster-admin") | {
    name: .metadata.name,
    subjects: [.subjects[]? | "\(.kind)/\(.name)"]
  }'
```

실측 결과 (minikube)

```
{ "name": "cluster-admin",          "subjects": ["Group/system:masters"] }
{ "name": "kubeadm:cluster-admins", "subjects": ["Group/kubeadm:cluster-admins"] }
{ "name": "minikube-rbac",          "subjects": ["ServiceAccount/default"] } ⚠
```

즉시 점검 신호

- `Group/system:masters` / `kubeadm:cluster-admins` 외 항목은 모두 점검 대상
- `minikube-rbac` → `ServiceAccount/default` 발견 — 책에서 경고한 정확히 그 패턴이 minikube 기본값에 존재 (학습 환경이라 OK, 운영에선 사고)

사고 사례 — Tesla 2018

사건

- Tesla Kubernetes 대시보드가 인증 없이 인터넷 공개
- 공격자가 대시보드 진입 후 클러스터 컨트롤 획득
- AWS 자격증명을 가진 Pod 발견 → 암호화폐 채굴

학습 포인트

- 대시보드/관리 UI에 인증 미설정 = 신원 단계 자체가 비어있음
- 클러스터 안의 Secret/자격증명은 클러스터 권한 = 외부 권한으로 연결됨
- ServiceAccount에 cloud IAM이 연결돼 있다면 RBAC 사고 = 클라우드 사고

최소 권한 원칙 — 체크리스트

SA 단위

- 워크로드마다 전용 SA (default SA 사용 금지)
- API 호출 안 하는 Pod엔 `automountServiceAccountToken: false`
- 토큰 audience 명시

RBAC 단위

- Role 정의 시 `*` 와일드카드 금지 (verb/resource 명시)
- ClusterRoleBinding은 정말 필요할 때만 (감사 대상)
- cluster-admin은 사람에게만, SA에는 절대 부여하지 않음

운영 단위

- 권한 변경 시 `kubectl auth can-i --as=...` 로 사전 검증
- 정기적으로 cluster-admin binding 점검

책 1판 vs 현행 — 12장 전체 정리

주제	책 1판 (2018)	현행 (2026, 1.30+)
SA 토큰 발급	Secret에 영구 토큰 자동 생성	1.24+ 자동 생성 안 함
토큰 수명	무제한	1시간 기본, 회전
토큰 마운트	Secret 볼륨	Projected Volume
audience	없음	명시 가능
Pod 보안	PodSecurityPolicy (PSP)	PSP 제거 (1.25) → PSA
사용자 인증	x509 / 정적 토큰	OIDC + IAM 통합 표준
RBAC 자체	<code>rbac.authorization.k8s.io/v1</code>	동일 (변경 없음)

핵심 메시지

RBAC는 안정. 토큰과 PSP는 큰 변화 — 책 따라가지 말고 현행 기준으로 운영

13장 예고 — Securing cluster nodes

12장이 API 호출 경로의 보안이었다면, 13장은 워크로드 실행 보안:

- Pod에 호스트 namespace 노출 (network/PID/IPC)
- Container Security Context (runAsUser, privileged 등)
- Pod Security Policy → Pod Security Admission (PSA)
- NetworkPolicy로 Pod 간 트래픽 격리

12장과의 연결

누구인지 + 무엇을 할 수 있는지를 정한 다음, 그 Pod이 노드 자체에 무엇을 할 수 있는가가 13장

Summary (1/2)

12.1 Authentication

- User는 K8s 밖, ServiceAccount는 K8s 안
- default SA는 자동 부착되지만 운영에선 전용 SA 사용
- 현행: Projected Volume + 1시간 토큰 자동 회전

12.2 Authorization (RBAC)

- 4종 리소스: Role / RoleBinding / ClusterRole / ClusterRoleBinding
- 권한은 verb × resource × apiGroup의 3차원
- Default deny + 화이트리스트 합집합 모델

Summary (2/2)

실무 운영 원칙

- 워크로드마다 전용 SA, cluster-admin은 SA에 부여 금지
- `kubectl auth can-i --as=...` 로 변경 사전 검증
- 정기적으로 ClusterRoleBinding 감사

한 줄 결론

K8s 보안의 첫 번째 방어선은 RBAC. 두 번째는 PSA + NetworkPolicy (13장).

Q&A

12장 — Securing the Kubernetes API Server

Authentication × RBAC